# Data Science

## Lecture 06: Decision Trees and Ensembles

Prof. Dr. Michael Granitzer

# Where We Left Off

Lectures 04–05 gave us models that predict and cannot explain themselves — a few hundred weights, or millions. Today: a model you can read aloud, and what it costs to keep it readable.

**Lecture 05**
accurate, opaque

→

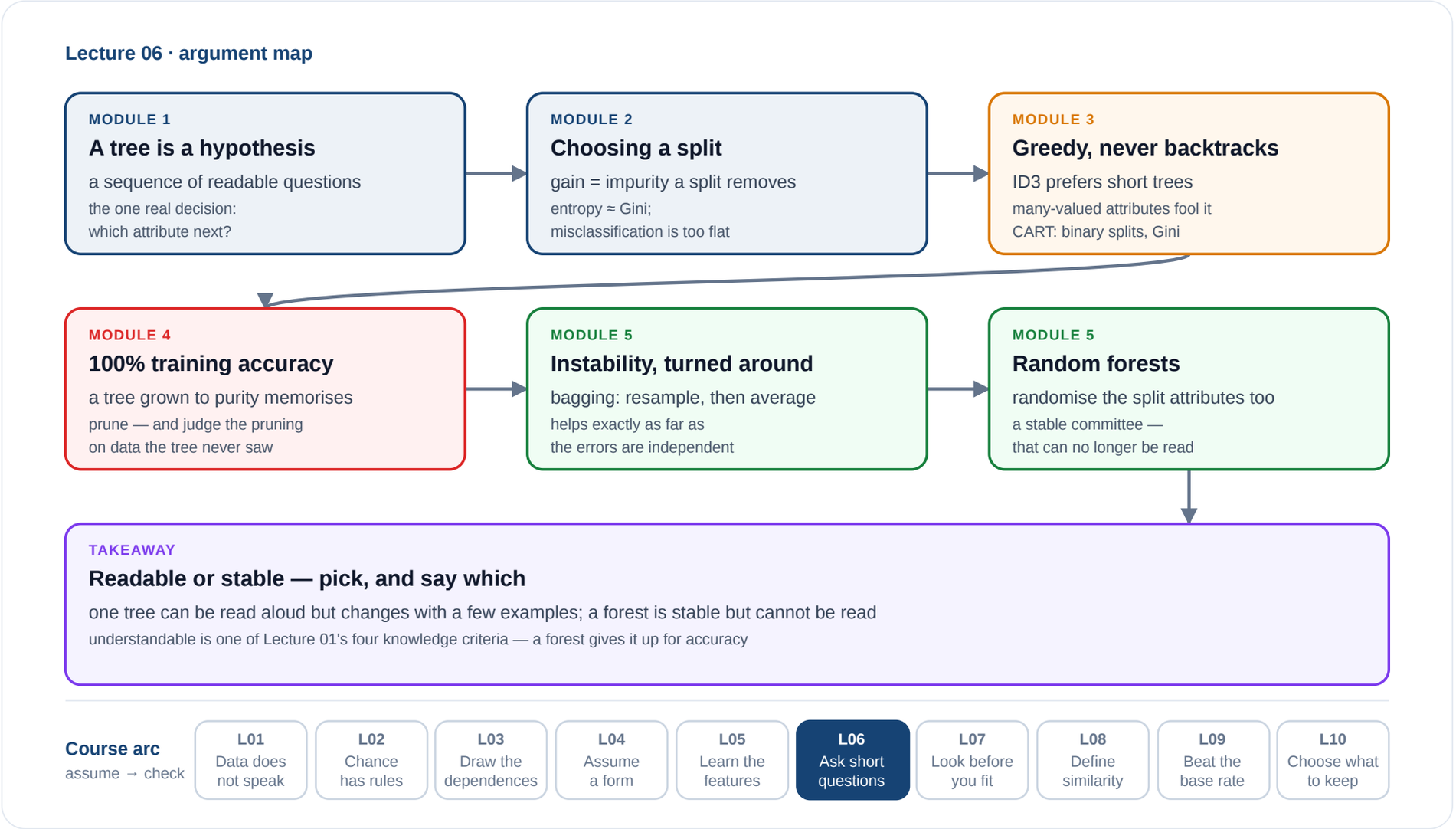
**Modules 1–4**
one tree: readable, unstable

→

**Module 5**
a forest: stable, opaque again

> "Ultimately understandable" was one of Lecture 01's four criteria for knowledge. This lecture is the first to trade it away on purpose.

# Argument Map



# Module 1

## Trees as Hypotheses

> ***Guiding Question:*** *A linear model hides its reasoning in 200 weights. What would a model look like that can be read aloud?*

# A Tree Is a Sequence of Questions

Each inner node tests one attribute, each branch is an answer, each leaf is a class.

| Example | Sky | Temperature | Humidity | Wind | EnjoySport |
|---|---|---|---|---|---|
| 1 | sunny | warm | normal | strong | yes |
| 2 | sunny | warm | high | strong | yes |
| 3 | rainy | cold | high | strong | **no** |
| 4 | sunny | warm | high | strong | yes |

To classify a new day, start at the root and follow the answers to a leaf.

> The tree **is** the hypothesis. Unlike a weight vector, it can be read aloud as a set of rules — which is why trees are the standard choice when a decision has to be explained.

# Construction and Classification

### Construction *(recursive)*

If the data is pure enough, make a leaf. Otherwise: choose a **split criterion**, decompose $D$ into $D_1, \ldots, D_s$, and recurse on each part.

### Classification *(recursive)*

If the node is a leaf, return its class. Otherwise: follow the successor selected by the example's attribute value.

The whole learning problem reduces to one question: **which attribute to split on?** Everything else is bookkeeping.

# When to Use Them

### Well suited

Attribute-value data; discrete or continuous features; missing values; a need for an explainable model; no assumption of linearity.

### Poorly suited

Smooth relationships (a tree approximates a diagonal boundary with a staircase); many equally relevant features; and — as we will see — stability.

# Mini-Exercise

Find a tree with a **single test** that classifies the four EnjoySport examples from the first slide correctly. Is it the only one?



# Mini-Exercise

Find a tree with a **single test** that classifies the four EnjoySport examples from the first slide correctly. Is it the only one?

> `Sky` works (sunny → yes, rainy → no) — and so does `Temperature` (warm → yes, cold → no). Both separate the data perfectly; even the next module's split criterion scores them equally (0.811 bits each). The data cannot choose: the learner's **inductive bias**, not the examples, breaks the tie.

# Takeaway

A decision tree is a **readable hypothesis** built by recursive splitting. The only real decision is which attribute to split on next — and the data alone does not settle it.

# Module 2

## Choosing a Split

> ***Guiding Question:*** *Every attribute could be split on. What makes one split better than another — and how do we measure it?*

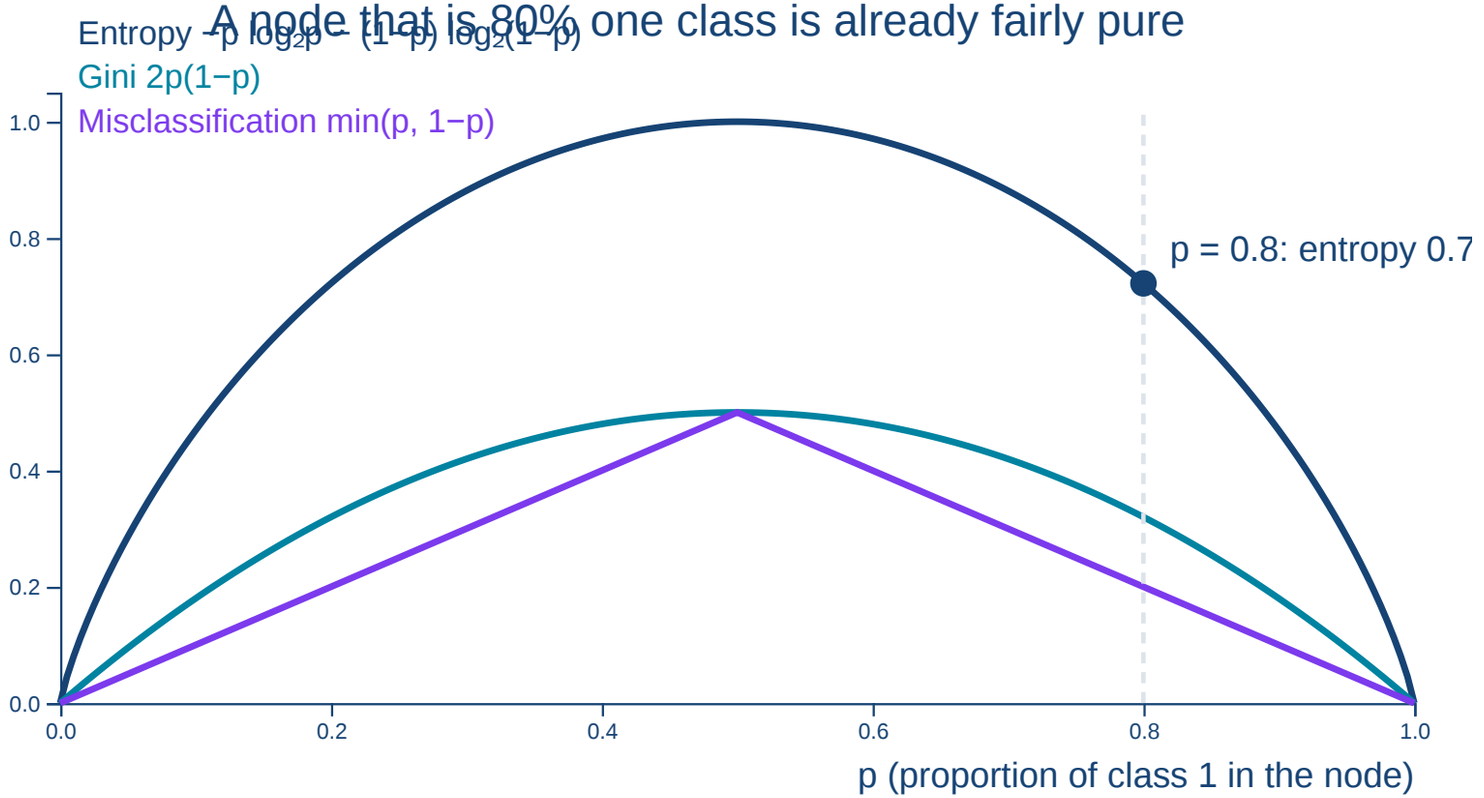
# What Makes a Split Good?

A split is good if the resulting subsets are **purer** than what we started with — closer to containing a single class.

So we need a number that measures impurity: maximal when the classes are evenly mixed, zero when a node is pure.

# Three Measures

A node that is 80% one class is already fairly pure



# Entropy and Gini

For a node with class probabilities $P(A_1), \ldots, P(A_k)$:

$$H(\mathcal{A}) = -\sum_{i=1}^{k} P(A_i) \log_2 P(A_i) \qquad G(\mathcal{A}) = 1 - \sum_{i=1}^{k} P(A_i)^2$$

**Entropy** $H$ is measured in bits — the average number of yes/no questions needed to identify an example's class. **Gini impurity** $G$ is the probability that two examples drawn at random from the node belong to different classes.

| | Pure node | Two classes, 50/50 | $k$ equal classes |
|---|---|---|---|
| Entropy $H$ | 0 | 1 bit | $\log_2 k$ bits |
| Gini $G$ | 0 | 0.5 | $1 - 1/k$ |

# Conditional Entropy and Information Gain

After splitting on an attribute $\mathcal{B}$ with outcomes $B_1, \ldots, B_s$:

$$H(\mathcal{A} \mid \mathcal{B}) = \sum_{j=1}^{s} P(B_j) \cdot H(\mathcal{A} \mid B_j)$$

— the impurity of each part, weighted by how much data lands in it.

$$\text{Gain}(\mathcal{B}) = H(\mathcal{A}) - H(\mathcal{A} \mid \mathcal{B})$$

**Information gain** is the impurity removed by the split. Choose the attribute that maximises it.

# Worked Example: The Root of PlayTennis

Quinlan's 14 weather days: 9 yes, 5 no, so $H = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.940$ bits. Counts are (yes, no) per value.

| Attribute | Split | $H(\mathcal{A} \mid \mathcal{B})$ | Gain |
|---|---|---|---|
| **Outlook** | sunny (2, 3) · overcast (4, 0) · rain (3, 2) | 0.694 | **0.247** |
| Humidity | high (3, 4) · normal (6, 1) | 0.788 | 0.152 |
| Wind | weak (6, 2) · strong (3, 3) | 0.892 | 0.048 |
| Temperature | hot (2, 2) · mild (4, 2) · cool (3, 1) | 0.911 | 0.029 |

# Worked Example: The Root of PlayTennis

Quinlan's 14 weather days: 9 yes, 5 no, so $H = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.940$ bits. Counts are (yes, no) per value.

| Attribute | Split | $H(\mathcal{A} \mid \mathcal{B})$ | Gain |
|---|---|---|---|
| **Outlook** | sunny (2, 3) · overcast (4, 0) · rain (3, 2) | 0.694 | **0.247** |
| Humidity | high (3, 4) · normal (6, 1) | 0.788 | 0.152 |
| Wind | weak (6, 2) · strong (3, 3) | 0.892 | 0.048 |
| Temperature | hot (2, 2) · mild (4, 2) · cool (3, 1) | 0.911 | 0.029 |

> Outlook wins the root although two of its branches are *more* mixed than the root (0.971 vs 0.940 bits): its overcast branch is **pure**, so 4 of the 14 days need no further question.

# Mini-Exercise

A node holds 4 yes and 4 no. A split yields [3 yes, 1 no] and [1 yes, 3 no]; a 3:1 node has entropy 0.811. What is the information gain?



# Mini-Exercise

A node holds 4 yes and 4 no. A split yields [3 yes, 1 no] and [1 yes, 3 no]; a 3:1 node has entropy 0.811. What is the information gain?

> Before: $H = 1$ bit. After: both halves hold half the data and have $H = 0.811$, so $H(\mathcal{A} \mid \mathcal{B}) = 0.811$. $\mathrm{Gain} = 1 - 0.811 = 0.189$ bits — a split that "looks decent" removes less than a fifth of a bit.

# Takeaway

Impurity measures how mixed a node is; **information gain is the impurity a split removes**, weighted by the data it moves. Entropy and Gini agree closely; misclassification error is too flat to choose between splits.

# Module 3

## Building the Tree

> ***Guiding Question:*** *ID3 never reconsiders a split. What does that greedy choice assume about the world?*

# ID3

**ID3**(examples $D$, attributes $A$, target)

1. If all examples share one class $\rightarrow$ return a leaf with that class.
2. If $A$ is empty $\rightarrow$ return a leaf with the majority class.
3. Otherwise choose $A^* \in A$ with the highest **information gain**.
4. For each value $a$ of $A^*$: if $D_a$ is empty, add a leaf with the majority class of $D$; otherwise add the subtree **ID3**($D_a$, $A \setminus \{A^*\}$, target).

Greedy and recursive: each split is chosen for immediate gain, and never reconsidered.

# What ID3 Assumes

### Hypothesis space

**All** decision trees over the attributes — so the target function is certainly in there. No representational bias.

### Search bias

It prefers trees that place **high-gain attributes near the root**, and therefore **shorter trees**. It stops at the first tree that fits.

Two assumptions, not facts. **Short trees generalise** — Occam's razor as an algorithm. And **a useful attribute shows gain on its own**: for $y = x_1 \operatorname{XOR} x_2$ on balanced data, each attribute alone has gain 0, yet together they decide $y$ — greedy search cannot see it.

# CART

## What differs

**Binary** splits only; **Gini** impurity instead of entropy; handles **regression** by splitting on variance reduction and predicting the mean in each leaf.

## What does not

The template is unchanged: choose a split, decompose, recurse. CART is a different set of choices inside the same skeleton.

Binary splits reduce a defect of multi-way splits: an attribute with many values (an ID number, in the extreme) achieves high information gain by fragmenting the data into pure singletons. C4.5 corrects for it with the **gain ratio**, the gain divided by the entropy of the split itself.

# Mini-Exercise

Your dataset has a `customer_id` column, unique for every row. What information gain does a split on it achieve — and what does the resulting tree predict for a new customer?

# Mini-Exercise

Your dataset has a `customer_id` column, unique for every row. What information gain does a split on it achieve — and what does the resulting tree predict for a new customer?

> The **maximum possible**: every branch holds one example, every leaf is pure, so the gain equals the root's entropy. And for a new customer — nothing: their id matches no branch. Perfect purity, zero generalisation; hence gain ratio, or binary splits.

# Takeaway

ID3 is a **greedy, non-backtracking search** whose bias is a preference for short trees. CART changes the split type and the impurity measure, not the idea. Both can be fooled by an attribute with too many values.

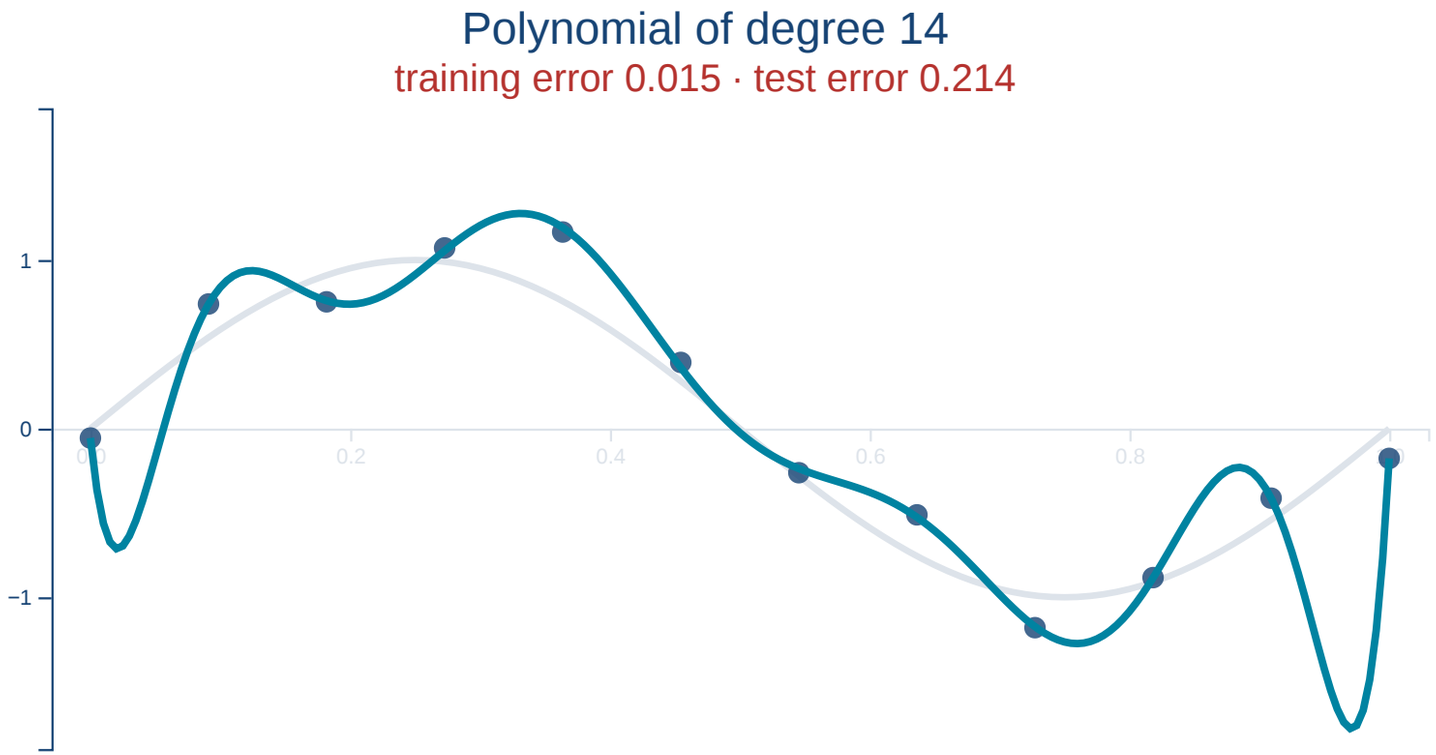
# Module 4

## Overfitting and Pruning

> ***Guiding Question:*** *A tree can reach 100% training accuracy on any dataset without contradictory examples. Why is that a symptom rather than an achievement?*

# A Tree Grown to Purity Is Overfitted

Left alone, ID3 splits until every leaf is pure — memorising the training set, noise included.




Tree depth plays the role of polynomial degree: more splits, lower training error, and eventually rising test error.

# Two Places to Intervene

### Stopping (pre-pruning)

Halt splitting **during** construction — when a node is small, when gain is below a threshold, or at a depth limit.

Cheap, but short-sighted: a weak split may enable a strong one beneath it.

### Pruning (post-pruning)

Grow the full tree, then **remove** subtrees that do not earn their keep.

More expensive, and generally more reliable.

# Reduced Error Pruning

1. Split the data into a **training** and a **validation** set.
2. Grow the full tree on the training set.
3. For each inner node: consider replacing its subtree with a leaf carrying the majority class.
4. Keep the replacement if accuracy on the **validation** set does not fall.
5. Repeat until no beneficial replacement remains.

The decision is made on data the tree was **not** grown on. Pruning against the training set would remove nothing — every subtree helps there by construction.

# Mini-Exercise

A tree with 340 leaves scores 100% on training data, 78% on validation. Pruned to 41 leaves it scores 94% and 89%. Training accuracy fell by six points — did pruning fail?

# Mini-Exercise

A tree with 340 leaves scores 100% on training data, 78% on validation. Pruned to 41 leaves it scores 94% and 89%. Training accuracy fell by six points — did pruning fail?

> No. The 299 removed leaves fitted noise — examples that are now misclassified, as intended. Unseen data gained **11 points**, and the tree became small enough to read. Training accuracy was never the objective.

# Takeaway

> An unpruned tree memorises. **Pruning trades training accuracy for generalisation**, and the trade must be judged on data the tree never saw.

# Module 5

## Ensembles

> ***Guiding Question:*** *Change a few examples and the tree changes completely. Can that instability be turned into an advantage?*

# Why One Learner Falls Short

Three reasons, after Dietterich (2000):

### Statistical

Too little data for the hypothesis space: many hypotheses fit equally well, and small changes in the data pick a very different one.

### Computational

The learner cannot search its hypothesis space exhaustively and settles for a heuristic path — a local optimum.

### Representational

The model family cannot express the target at all; a combination of models may.

**Decision trees are the textbook unstable learner** (the statistical reason): change a handful of examples and a different attribute wins at the root, producing a completely different tree.

# The Idea

**Ensemble learning** builds several learners and combines their predictions — by majority vote for classification, by averaging for regression.

### Bagging

**B**ootstrap **agg**regat**ing** — train on resampled datasets.

### Boosting

Train weak learners in sequence, each focusing on the previous one's mistakes. *Named here; not developed in this course.*

### Random forest

Bagging plus randomised attribute selection at every split.

Ensembles exploit instability: **many different models, each better than chance and wrong in partly independent ways, are right together more often than any one of them**.

# Bootstrap Aggregating

**The problem.** A group of predictors helps only if they differ — but we have one algorithm and one dataset, which yield one model.

**The solution.** Manufacture different datasets: draw $n$ examples from $D$ **with replacement**, train a predictor on each sample, and combine them by vote or average.

> The learner never changes. Only the data it sees does.

# What Resampling Actually Does

Each bootstrap sample contains about **63.2%** of the distinct examples of $D$. The probability that a particular example is *never* drawn in $n$ tries is

$$\left(1 - \frac{1}{n}\right)^n \xrightarrow[n\to\infty]{} e^{-1} \approx 0.368$$

### The gain

Each model sees a different two-thirds of the data, so their errors are partly independent. **Averaging reduces variance** — and therefore overfitting.

### The left-out third

The ~36.8% not drawn is the *out-of-bag* sample, usable as a free validation set for that model.

# Mini-Exercise

You bag 100 trees, each 70% accurate, and take a majority vote. What accuracy does the vote reach if all 100 trees are identical — and if their errors are independent?

# Mini-Exercise

You bag 100 trees, each 70% accurate, and take a majority vote. What accuracy does the vote reach if all 100 trees are identical — and if their errors are independent?

> Identical: **70%** — a hundred copies vote unanimously. Independent: the vote is wrong only if 50 or more trees err at once, which happens with probability $2.2 \times 10^{-5}$ — about **99.998%**. Both conditions matter: each tree better than chance (> 50%), and errors that are not shared.

# Takeaway

> Bagging manufactures diversity by **resampling the data**, and averages away variance. It helps exactly to the extent that the models' errors are independent — and not at all when they are not.

# Why Bagging Trees Is Not Enough

### Trees stay similar

If one attribute has much higher gain than the rest, it wins at the root of **every** bootstrap sample. The trees correlate, and averaging them buys little.

### And it costs

Interpretability is gone — you can no longer read the model as a set of rules — while the computation of 100 near-identical trees is largely wasted.

# The Fix: Randomise the Split Too

**Random forest** (Breiman 2001)

For each of $T$ trees:

1. Draw a bootstrap sample from $D$.
2. Grow a tree — but **at every node**, choose the split from a random subset $A' \subset A$ of $m_A$ attributes rather than from all of them.
3. Do not prune.

Combine by majority vote (classification) or average (regression).

The dominant attribute is simply **absent** from most nodes' candidate sets, so other attributes get to define the tree. Diversity is engineered rather than hoped for.

# What You Get and What You Give Up

### Gained

Strong accuracy with almost no tuning; out-of-bag error for free; a usable attribute-importance measure.

### Lost

The readable single tree. A forest is a **committee**, and no member speaks for it.

Attribute importance from a forest is not a causal statement. It reports which attributes the trees found useful for splitting — under this data, this sampling, this correlation structure.

# Mini-Exercise

A pruned tree reaches 86% accuracy, a random forest 91%. Which do you deploy for an internal churn model — and which for automated loan refusals that must be justified to applicants?

# Mini-Exercise

A pruned tree reaches 86% accuracy, a random forest 91%. Which do you deploy for an internal churn model — and which for automated loan refusals that must be justified to applicants?

> Churn: the **forest** — five points matter and nobody needs the model explained. Loans: the **tree**, or a forest inside a documented, auditable process; a refusal that cannot be explained may be neither acceptable nor lawful. The best model fits the **decision**, not the leaderboard.

# Takeaway

Random forests **engineer diversity** by randomising the attributes available at each split, turning unstable trees into a stable committee — at the cost of the one thing a single tree was best at: being readable.

# The Churn Shop: A Tree You Can Read Aloud

**ONE TREE**

→ Contract **monthly**?

→ No **app login** for 30 days?

→ Then: **likely to churn** — call them.

The retention team can check every step against what they know.

**A FOREST OF 500 TREES**

Usually more accurate, and stable when a few customers change.

Its answer is a vote. Asked *why* this customer, it has no sentence to offer.

The same data, two answers to Lecture 01's criteria: the tree is **understandable** and fragile, the forest is accurate and unreadable. Say which one you chose, and why.